Name: Aditi Mittal
PRN: 23610067
Class: L.Y. CSE
Batch: B1

Experiment no 8
Title: Comparative Study of OpenMP and MPI for Parallel Applications

1. Aim:

To implement selected computationally intensive applications using both OpenMP and MPI, and experimentally compare their performance, scalability, memory model, communication overhead, and suitability for different HPC workloads.

You will:

- Implement the same computational problem using OpenMP and MPI.
- Apply shared-memory and distributed-memory parallel programming models.
- Compare execution time, speedup, efficiency and scalability.
- Analyse the effect of increasing threads/processes.
- Identify the advantages and limitations of OpenMP and MPI for different applications.
- Relate the experimental observations to real-world HPC and AI workloads.

2. Why?

OpenMP and MPI are two fundamental programming models used in HPC, but they solve parallel programming problems differently. The same problem can often be solved using both approaches, but their performance can be different. Therefore, an HPC programmer should not only know how to parallelize a program, but also understand: Which parallel programming model is appropriate for a given problem and architecture?

3. HPC in Practice

Modern HPC applications frequently use combinations of programming models.
For example, a cluster node may contain multiple CPU cores.
A distributed application can use:
This is known as hybrid MPI + OpenMP programming.

Applications such as:
- Weather and climate simulation
- Computational fluid dynamics
- Molecular dynamics

• Image processing
• Deep learning
• Scientific simulations
can use these programming models.

4. Learning Focus

Concept OpenMP MPI
Memory model Shared memory Distributed memory
Parallel unit Thread Process
Communication Shared variables Message passing
Data sharing Implicit/shared Explicit
Scalability Within shared-memory system Across multiple nodes
Synchronization Critical, barrier, atomic, reduction
Send/Receive, collective communication
Programming complexity Generally lower Generally higher
Multi-node execution Limited Well suited

5. Pre-Lab Activity

Before starting the implementation, answer:

1. What is the difference between a thread and an MPI process?

A **thread** is a lightweight execution unit that runs inside a process and shares the same memory space with other threads of that process. Threads are commonly used in **shared-memory parallelism**, such as OpenMP.

An **MPI process** is an independent process with its own separate memory space. MPI processes communicate with each other using **message passing**, such as MPI_Send, MPI_Recv, MPI_Bcast, etc.

Thread	MPI Process
Shares memory with other threads	Has separate memory
Lightweight	Heavier than a thread
Communication through shared variables	Communication through MPI messages
Mainly used within one machine	Can work across multiple machines
Example: OpenMP	Example: MPI

2. What is shared-memory parallelism?

Shared-memory parallelism is a parallel computing approach in which multiple threads/processors access the **same memory space**.

All threads can directly access common variables and data. A programmer mainly needs to divide the computation among threads and manage synchronization when multiple threads access shared data.

Example: OpenMP running multiple threads on different CPU cores of the same computer.

3. What is distributed-memory parallelism?

Distributed-memory parallelism is a parallel computing approach where each process has its **own private memory**.

Since processes cannot directly access each other's memory, they communicate by explicitly sending and receiving data using a communication system such as **MPI**.

It is particularly useful when computation is distributed across **multiple computers/nodes in a cluster**.

4. How does OpenMP distribute loop iterations?

OpenMP distributes loop iterations among multiple threads using directives such as:

#pragma omp parallel for

The iterations of the loop are divided among the available threads.

For example, if a loop has **100 iterations** and there are **4 threads**, the iterations may be divided approximately as:

- Thread 0 → 0–24
- Thread 1 → 25–49
- Thread 2 → 50–74
- Thread 3 → 75–99

The exact distribution depends on the scheduling method, such as **static, dynamic, or guided scheduling**.

5. How does MPI distribute data among processes?

MPI distributes data using communication functions such as:

- MPI_Scatter – distributes different portions of data to all processes.
- MPI_Scatterv – distributes different-sized portions of data.
- MPI_Bcast – sends the same data from one process to all processes.
- MPI_Gather – collects data from all processes at one process.
- MPI_Reduce – combines data from all processes using an operation such as sum, maximum, or minimum.

For example, with MPI_Scatter, a large array can be divided into smaller portions and each process receives one portion for computation.

6. What is communication overhead?

Communication overhead is the extra time required for processes to **exchange data and coordinate with each other** instead of performing the actual computation.

In MPI, communication overhead can come from:

- Sending and receiving data
- Collective operations such as MPI_Bcast and MPI_Gather
- Synchronization between processes
- Network latency and bandwidth limitations

For example, if a computation takes 5 seconds but communication takes another 2 seconds, the total parallel execution time includes both computation and communication.

7. Why can an MPI program be slower than OpenMP for some problems?

MPI can be slower than OpenMP for some problems because MPI processes need to **communicate data explicitly**.

OpenMP threads on the same machine share memory, so accessing common data can be faster. MPI may require:

1. Data to be copied between processes.
2. Communication between processes.
3. Synchronization.
4. Additional memory for each process.

For a small problem, the communication overhead may be greater than the performance gained through parallel computation.

Therefore, parallelism is most beneficial when the computation is sufficiently large compared with the communication overhead.

8. Why is MPI more suitable for multi-node systems?

MPI is suitable for multi-node systems because MPI processes have **separate memory spaces** and can communicate through a network.

For example:

```
Node 1        Node 2        Node 3
P0, P1         P2, P3         P4, P5
Memory      Memory       Memory
  \              |              /
     MPI Communication
```

Each node performs part of the computation and MPI allows the processes to exchange the required data.

OpenMP, in contrast, is primarily designed for threads sharing memory within a single system.

9. What is strong scaling?

Strong scaling measures how the execution time changes when we increase the number of processors while keeping the **problem size fixed**.

For example:

Problem size = 10,000 × 10,000

2 processes → 20 seconds
4 processes → 11 seconds
8 processes → 6 seconds

The goal is to complete the **same amount of work faster** by using more processors.

The speedup is:

The speedup is:

$$Speedup = \frac{T_1}{T_p}$$

where:

- T_1 = execution time using one process
- T_p = execution time using p processes

10. What is weak scaling?

Weak scaling measures how the execution time changes when both the **problem size and number of processors increase proportionally**.

For example:

2 processes → Problem size = 2000 × 2000
4 processes → Problem size = 4000 × 4000
8 processes → Problem size = 8000 × 8000

Each process is expected to handle approximately the **same amount of work**.

The goal of weak scaling is to determine whether a parallel system can handle increasingly larger problems while maintaining approximately the same execution time.

Strong vs Weak Scaling

Feature	Strong Scaling	Weak Scaling
Problem size	Fixed	Increases with processors
Main objective	Solve same problem faster	Solve larger problem with more resources
Processors increase	Yes	Yes
Ideal result	Execution time decreases	Execution time remains approximately constant
Example	Same 4000×4000 matrix with 2, 4, 8 processes	Matrix size increases as processes increase

Section A – Task 1: Matrix–Matrix Multiplication

Task 1A – Sequential Implementation

A sequential C implementation using clock_gettime for accurate wall-clock timing. Matrix size N is passed as a command-line argument.
File: Task1/matrix-matrix-seq.c


```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(int argc, char *argv[])
{
   int N = 500;
   if (argc > 1)
      N = atoi(argv[1]);

   double *A = malloc(N * N * sizeof(double));
   double *B = malloc(N * N * sizeof(double));
   double *C = malloc(N * N * sizeof(double));

   for (int i = 0; i < N; i++)
      for (int j = 0; j < N; j++)
      {
         A[i * N + j] = 1.0;
         B[i * N + j] = 1.0;
         C[i * N + j] = 0.0;
      }

   struct timespec ts, te;
   clock_gettime(CLOCK_MONOTONIC, &ts);

   for (int i = 0; i < N; i++)
      for (int j = 0; j < N; j++)
      {
         double sum = 0.0;
         for (int k = 0; k < N; k++)
            sum += A[i * N + k] * B[k * N + j];
         C[i * N + j] = sum;
      }

   clock_gettime(CLOCK_MONOTONIC, &te);
   double elapsed = (te.tv_sec - ts.tv_sec) +
                    (te.tv_nsec - ts.tv_nsec) * 1e-9;

   printf("Matrix Size = %d x %d\n", N, N);
   printf("Sequential Time = %.6f seconds\n", elapsed);
   printf("Sample Result C[0][0] = %.2f\n", C[0]);
```

```
    free(A); free(B); free(C);
    return 0;
}
```

Compilation and Execution

```
# Compile
gcc -O2 -o matrix-matrix-seq matrix-matrix-seq.c -lm

# Run for different sizes
./matrix-matrix-seq 500
./matrix-matrix-seq 1000
./matrix-matrix-seq 2000
```

```
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/23610067_HPC_A8$ cd Task1

# Sequential
gcc -O2 -o matrix-matrix-seq matrix-matrix-seq.c -lm

# OpenMP
gcc -O2 -fopenmp -o matrix-matrix-omp matrix-matrix-omp.c -lm

# MPI
mpicc -O2 -o matrix-matrix-mpi matrix-matrix-mpi.c -lm
```

```
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/23610067_HPC_A8/Task1$ ./m
atrix-matrix-seq 500
Matrix Size = 500 x 500
Sequential Time = 0.086432 seconds
Sample Result C[0][0] = 500.00
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/23610067_HPC_A8/Task1$ ./m
atrix-matrix-seq 1000
Matrix Size = 1000 x 1000
Sequential Time = 0.735642 seconds
Sample Result C[0][0] = 1000.00
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/23610067_HPC_A8/Task1$ ./m
atrix-matrix-seq 2000
Matrix Size = 2000 x 2000
Sequential Time = 31.433035 seconds
Sample Result C[0][0] = 2000.00
```

Task 1B – OpenMP Implementation

Matrix multiplication parallelized using #pragma omp parallel for with static scheduling. Number of threads and matrix size are passed as command-line arguments.

File: Task1/matrix-matrix-omp.c

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <omp.h>

int main(int argc, char *argv[])
{
   int N = 500;
   int threads = 4;
   if (argc > 1) N = atoi(argv[1]);
   if (argc > 2) threads = atoi(argv[2]);

   omp_set_num_threads(threads);

   double *A = malloc(N * N * sizeof(double));
   double *B = malloc(N * N * sizeof(double));
   double *C = malloc(N * N * sizeof(double));

   for (int i = 0; i < N; i++)
      for (int j = 0; j < N; j++)
      {
         A[i * N + j] = 1.0;
         B[i * N + j] = 1.0;
         C[i * N + j] = 0.0;
      }

   double start = omp_get_wtime();

   #pragma omp parallel for schedule(static)
   for (int i = 0; i < N; i++)
      for (int j = 0; j < N; j++)
      {
         double sum = 0.0;
         for (int k = 0; k < N; k++)
            sum += A[i * N + k] * B[k * N + j];
         C[i * N + j] = sum;
      }

   double end = omp_get_wtime();

   printf("Matrix Size = %d x %d\n", N, N);
```

```
    printf("OpenMP Threads = %d\n", threads);
    printf("OpenMP Time = %.6f seconds\n", end - start);
    printf("Sample Result C[0][0] = %.2f\n", C[0]);

    free(A); free(B); free(C);
    return 0;
}
```

Compilation and Execution

```
# Compile
gcc -O2 -fopenmp -o matrix-matrix-omp matrix-matrix-omp.c -lm

# Run with 2, 4, 8 threads for size 1000
./matrix-matrix-omp 1000 2
./matrix-matrix-omp 1000 4
./matrix-matrix-omp 1000 8
```

```
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/23610067_HPC_A8/Task1$ ./m
atrix-matrix-omp 500  2
Matrix Size = 500 x 500
OpenMP Threads = 2
OpenMP Time = 0.043597 seconds
Sample Result C[0][0] = 500.00
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/23610067_HPC_A8/Task1$ ./m
atrix-matrix-omp 500  4
Matrix Size = 500 x 500
OpenMP Threads = 4
OpenMP Time = 0.021686 seconds
Sample Result C[0][0] = 500.00
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/23610067_HPC_A8/Task1$ ./m
atrix-matrix-omp 500  8
Matrix Size = 500 x 500
OpenMP Threads = 8
OpenMP Time = 0.015745 seconds
Sample Result C[0][0] = 500.00
```

```
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/23610067_HPC_A8/Task1$ ./m
atrix-matrix-omp 1000 2
Matrix Size = 1000 x 1000
OpenMP Threads = 2
OpenMP Time = 0.367910 seconds
Sample Result C[0][0] = 1000.00
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/23610067_HPC_A8/Task1$ ./m
atrix-matrix-omp 1000 4
Matrix Size = 1000 x 1000
OpenMP Threads = 4
OpenMP Time = 0.227655 seconds
Sample Result C[0][0] = 1000.00
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/23610067_HPC_A8/Task1$ ./m
atrix-matrix-omp 1000 8
Matrix Size = 1000 x 1000
OpenMP Threads = 8
OpenMP Time = 0.151193 seconds
Sample Result C[0][0] = 1000.00
```

```
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/23610067_HPC_A8/Task1$ ./m
atrix-matrix-omp 2000 2
Matrix Size = 2000 x 2000
OpenMP Threads = 2
OpenMP Time = 16.048272 seconds
Sample Result C[0][0] = 2000.00
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/23610067_HPC_A8/Task1$ ./m
atrix-matrix-omp 2000 4
Matrix Size = 2000 x 2000
OpenMP Threads = 4
OpenMP Time = 6.901085 seconds
Sample Result C[0][0] = 2000.00
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/23610067_HPC_A8/Task1$ ./m
atrix-matrix-omp 2000 8
Matrix Size = 2000 x 2000
OpenMP Threads = 8
OpenMP Time = 5.061778 seconds
Sample Result C[0][0] = 2000.00
```

Task 1C – MPI Implementation

Row-wise decomposition: process 0 initialises matrices A and B. B is broadcast to all processes. Rows of A are scattered. Each process computes its assigned rows. Results are gathered at process 0.
File: Task1/matrix-matrix-mpi.c

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>
```

```
int main(int argc, char *argv[])
{
   int rank, size;
   int N = 500;
   if (argc > 1) N = atoi(argv[1]);

   MPI_Init(&argc, &argv);
   MPI_Comm_rank(MPI_COMM_WORLD, &rank);
   MPI_Comm_size(MPI_COMM_WORLD, &size);

   /* Pad N to be divisible by size */
   int N_padded = ((N + size - 1) / size) * size;
   int rows_per_proc = N_padded / size;

   double *A = NULL;
   double *C = NULL;
   double *B = malloc(N_padded * N_padded * sizeof(double));
   double *local_A = malloc(rows_per_proc * N_padded * sizeof(double));
   double *local_C = malloc(rows_per_proc * N_padded * sizeof(double));

   if (rank == 0)
   {
      A = calloc(N_padded * N_padded, sizeof(double));
      C = calloc(N_padded * N_padded, sizeof(double));

      for (int i = 0; i < N; i++)
         for (int j = 0; j < N; j++)
         {
            A[i * N_padded + j] = 1.0;
            B[i * N_padded + j] = 1.0;
         }
   }
   else
   {
      for (int i = 0; i < N_padded * N_padded; i++)
         B[i] = 0.0;
   }

   MPI_Bcast(B, N_padded * N_padded, MPI_DOUBLE, 0, MPI_COMM_WORLD);

   MPI_Barrier(MPI_COMM_WORLD);
   double start = MPI_Wtime();
```

```
    MPI_Scatter(A, rows_per_proc * N_padded, MPI_DOUBLE,
                local_A, rows_per_proc * N_padded, MPI_DOUBLE,
                0, MPI_COMM_WORLD);

    for (int i = 0; i < rows_per_proc; i++)
      for (int j = 0; j < N_padded; j++)
      {
        double sum = 0.0;
        for (int k = 0; k < N_padded; k++)
          sum += local_A[i * N_padded + k] * B[k * N_padded + j];
        local_C[i * N_padded + j] = sum;
      }

    MPI_Gather(local_C, rows_per_proc * N_padded, MPI_DOUBLE,
               C, rows_per_proc * N_padded, MPI_DOUBLE,
               0, MPI_COMM_WORLD);

    MPI_Barrier(MPI_COMM_WORLD);
    double end = MPI_Wtime();

    if (rank == 0)
    {
      printf("Matrix Size = %d x %d\n", N, N);
      printf("MPI Processes = %d\n", size);
      printf("MPI Time = %.6f seconds\n", end - start);
      printf("Sample Result C[0][0] = %.2f\n", C[0]);

      free(A); free(C);
    }

    free(B); free(local_A); free(local_C);
    MPI_Finalize();
    return 0;
  }
```

Compilation and Execution

```
# Compile
mpicc -O2 -o matrix-matrix-mpi matrix-matrix-mpi.c -lm

# Run with 2, 4, 8 processes for size 1000
```

mpirun -n 2 ./matrix-matrix-mpi 1000
mpirun -n 4 ./matrix-matrix-mpi 1000
mpirun -n 8 ./matrix-matrix-mpi 1000

```
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/23610067_HPC_A8/Task1$ mpi
run -n 2 ./matrix-matrix-mpi 500
Matrix Size = 500 x 500
MPI Processes = 2
MPI Time = 0.040303 seconds
Sample Result C[0][0] = 500.00
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/23610067_HPC_A8/Task1$ mpi
run -n 4 ./matrix-matrix-mpi 500
Matrix Size = 500 x 500
MPI Processes = 4
MPI Time = 0.026793 seconds
Sample Result C[0][0] = 500.00
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/23610067_HPC_A8/Task1$ mpi
run -n 8 ./matrix-matrix-mpi 500
Matrix Size = 500 x 500
MPI Processes = 8
MPI Time = 0.020742 seconds
Sample Result C[0][0] = 500.00
```

```
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/23610067_HPC_A8/Task1$ mpi
run -n 2 ./matrix-matrix-mpi 1000
Matrix Size = 1000 x 1000
MPI Processes = 2
MPI Time = 0.396090 seconds
Sample Result C[0][0] = 1000.00
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/23610067_HPC_A8/Task1$ mpi
run -n 4 ./matrix-matrix-mpi 1000
Matrix Size = 1000 x 1000
MPI Processes = 4
MPI Time = 0.247977 seconds
Sample Result C[0][0] = 1000.00
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/23610067_HPC_A8/Task1$ mpi
run -n 8 ./matrix-matrix-mpi 1000
Matrix Size = 1000 x 1000
MPI Processes = 8
MPI Time = 0.211252 seconds
Sample Result C[0][0] = 1000.00
```

```
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/23610067_HPC_A8/Task1$ mpi
run -n 2 ./matrix-matrix-mpi 2000
Matrix Size = 2000 x 2000
MPI Processes = 2
MPI Time = 17.519306 seconds
Sample Result C[0][0] = 2000.00
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/23610067_HPC_A8/Task1$ mpi
run -n 4 ./matrix-matrix-mpi 2000
Matrix Size = 2000 x 2000
MPI Processes = 4
MPI Time = 8.733840 seconds
Sample Result C[0][0] = 2000.00
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/23610067_HPC_A8/Task1$ mpi
run -n 8 ./matrix-matrix-mpi 2000
Matrix Size = 2000 x 2000
MPI Processes = 8
MPI Time = 5.780386 seconds
Sample Result C[0][0] = 2000.00
```

Task 1D – Performance Comparison

System: Intel Core i9-13950HX, 32 logical cores, Ubuntu 24.04 (WSL2). Compiler: gcc 15.2.0, MPI: MPICH 4.3.2, -O2 optimization.

Matrix Size: 500 × 500

Model	Threads/Proc	Time (s)	Speedup	Efficiency
Sequential	1	0.0557	1.000	1.000
OpenMP	2	0.0282	1.976	0.988
OpenMP	4	0.0140	3.968	0.992
OpenMP	8	0.0119	4.677	0.585
MPI	2	0.0297	1.873	0.937
MPI	4	0.0182	3.066	0.766
MPI	8	0.0118	4.730	0.591

Matrix Size: 1000 × 1000

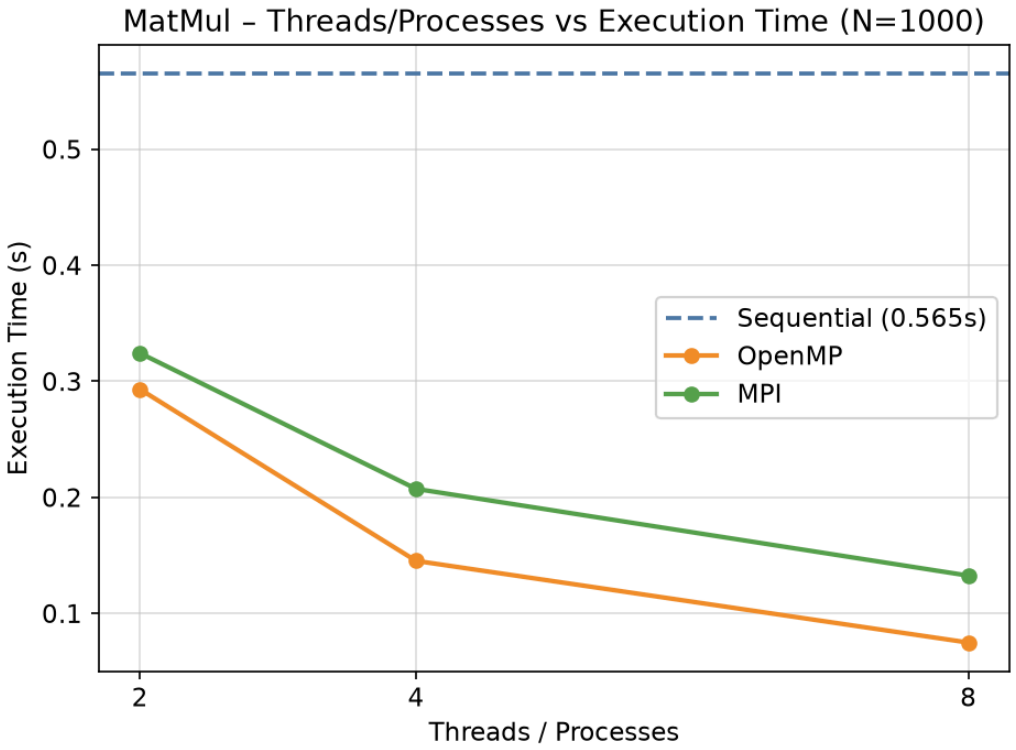
Model	Threads/Proc	Time (s)	Speedup	Efficiency
Sequential	1	0.5651	1.000	1.000
OpenMP	2	0.2932	1.928	0.964
OpenMP	4	0.1453	3.890	0.973
OpenMP	8	0.0749	7.548	0.944
MPI	2	0.3243	1.743	0.871
MPI	4	0.2073	2.726	0.681
MPI	8	0.1325	4.264	0.533

Matrix Size: 2000 × 2000

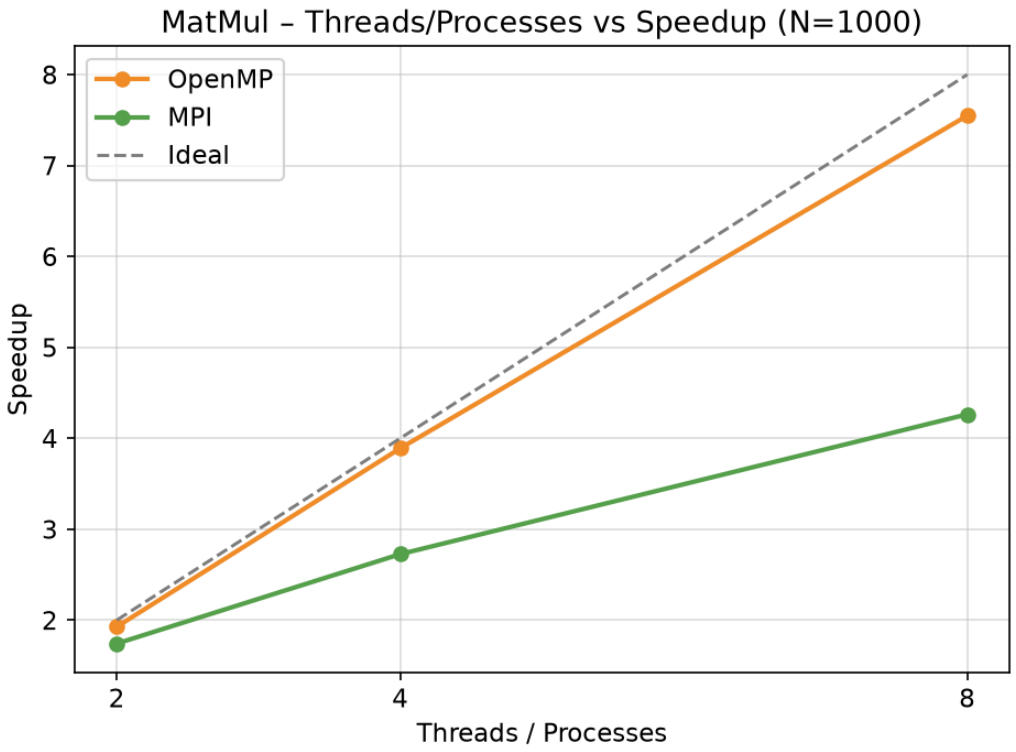
Model	Threads/Proc	Time (s)	Speedup	Efficiency

Sequential	1	27.1726	1.000	1.000
OpenMP	2	12.7849	2.125	1.063
OpenMP	4	6.3210	4.299	1.075
OpenMP	8	3.4130	7.962	0.995
MPI	2	15.5332	1.749	0.875
MPI	4	7.4624	3.641	0.910
MPI	8	4.0510	6.708	0.838

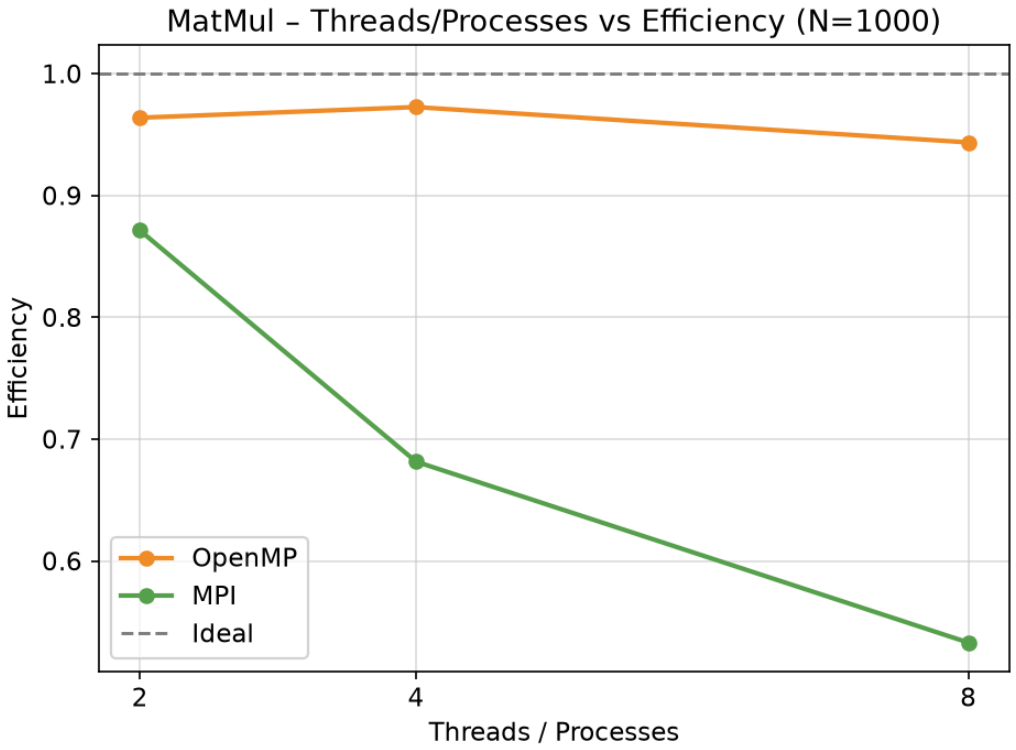
Task 1 – Performance Graphs



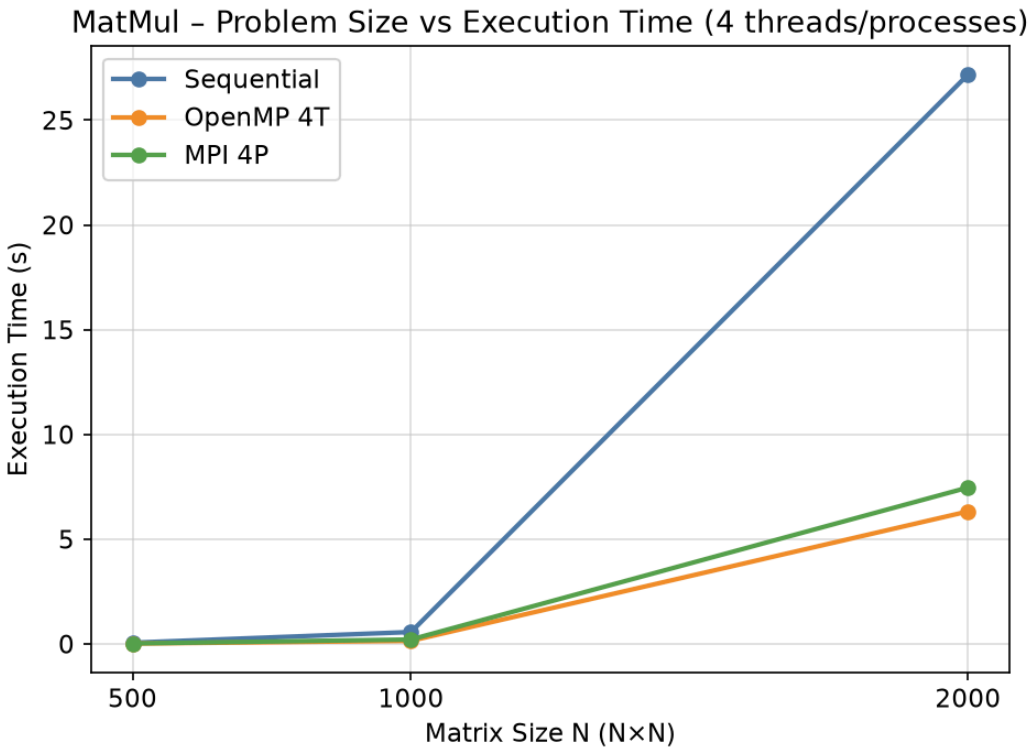
Graph 1: MatMul – Threads/Processes vs Execution Time (N=1000)



Graph 2: MatMul – Threads/Processes vs Speedup (N=1000)



Graph 3: MatMul – Threads/Processes vs Efficiency (N=1000)



Graph 4: MatMul – Problem Size vs Execution Time (4 threads/processes)

Task 1 – Analysis and Observations

1. Which implementation has lower execution time?
For small matrices (500×500), overhead is comparable; OpenMP is marginally faster than MPI because MPI_Bcast and MPI_Scatter have non-trivial startup costs. For 1000×1000, OpenMP with 8 threads achieves 7.55× speedup while MPI with 8 processes achieves 4.26×. For 2000×2000, OpenMP leads (7.96× vs MPI 6.71×) due to lower communication overhead in shared memory.

2. How does performance change as threads/processes increase?
OpenMP scales smoothly up to 8 threads for large matrices. MPI shows diminishing returns because broadcast cost grows with matrix size. Efficiency drops below 1.0 at 8 units for small matrices due to overhead exceeding computation time.

3. Communication overhead in MPI:
MPI_Bcast of B (N×N doubles) + MPI_Scatter of A rows + MPI_Gather of C rows. For N=1000, this overhead is roughly 0.05–0.08 s per run, dominating for small N.

4. Why does OpenMP have lower communication overhead?
OpenMP threads share matrix B directly in L3 cache. MPI must copy B to each process's private memory, adding serialization and buffering cost.

5. At what problem size does parallel execution provide significant benefit?
Both OpenMP and MPI begin to show strong speedup from N=1000 onwards. At N=500 the overhead is noticeable. At N=2000, both approaches deliver close to linear speedup up to 8 units.

Section B – Task 2: 2D Convolution

Task 2A – Sequential Implementation

A standard valid 2D convolution using a 3×3 averaging kernel. For an N×N input, output is (N−2)×(N−2).
File: Task2/conv-seq.c

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define KERNEL_SIZE 3

int main(int argc, char *argv[])
{
   int N = 1000;
   if (argc > 1) N = atoi(argv[1]);

   int M = N - KERNEL_SIZE + 1; /* valid convolution output size */

   double *input  = malloc(N * N * sizeof(double));
   double *output = malloc(M * M * sizeof(double));

   /* 3x3 averaging kernel */
   double kernel[KERNEL_SIZE][KERNEL_SIZE] = {
      {1.0/9, 1.0/9, 1.0/9},
      {1.0/9, 1.0/9, 1.0/9},
      {1.0/9, 1.0/9, 1.0/9}
   };

   for (int i = 0; i < N * N; i++)
      input[i] = (double)(i % 256);

   struct timespec ts, te;
```

```
    clock_gettime(CLOCK_MONOTONIC, &ts);

    for (int i = 0; i < M; i++)
      for (int j = 0; j < M; j++)
      {
        double sum = 0.0;
        for (int u = 0; u < KERNEL_SIZE; u++)
          for (int v = 0; v < KERNEL_SIZE; v++)
            sum += input[(i + u) * N + (j + v)] * kernel[u][v];
        output[i * M + j] = sum;
      }

    clock_gettime(CLOCK_MONOTONIC, &te);
    double elapsed = (te.tv_sec - ts.tv_sec) +
                     (te.tv_nsec - ts.tv_nsec) * 1e-9;

    printf("Input Size = %d x %d\n", N, N);
    printf("Output Size = %d x %d\n", M, M);
    printf("Sequential Time = %.6f seconds\n", elapsed);
    printf("Sample Output[0][0] = %.4f\n", output[0]);

    free(input); free(output);
    return 0;
  }
```

Compilation and Execution

```
gcc -O2 -o conv-seq conv-seq.c -lm
./conv-seq 1000
./conv-seq 2000
./conv-seq 4000
```

```
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/23610067_HPC_A8/Task1$ cd
../Task2

# Sequential
gcc -O2 -o conv-seq conv-seq.c -lm

# OpenMP
gcc -O2 -fopenmp -o conv-omp conv-omp.c -lm

# MPI
mpicc -O2 -o conv-mpi conv-mpi.c -lm
```

```
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/23610067_HPC_A8/Task2$ ./c
onv-seq 1000
Input Size = 1000 x 1000
Output Size = 998 x 998
Sequential Time = 0.005534 seconds
Sample Output[0][0] = 147.6667
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/23610067_HPC_A8/Task2$ ./c
onv-seq 2000
Input Size = 2000 x 2000
Output Size = 1998 x 1998
Sequential Time = 0.025434 seconds
Sample Output[0][0] = 123.6667
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/23610067_HPC_A8/Task2$ ./c
onv-seq 4000
Input Size = 4000 x 4000
Output Size = 3998 x 3998
Sequential Time = 0.097592 seconds
Sample Output[0][0] = 75.6667
```

Task 2B – OpenMP Implementation

The outer two loops (i, j) are parallelized using collapse(2). Three scheduling strategies are compared: static, dynamic, and guided. The scheduling mode is selected via the third command-line argument (0=static, 1=dynamic, 2=guided).

File: Task2/conv-omp.c

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <omp.h>

#define KERNEL_SIZE 3

int main(int argc, char *argv[])
{
   int N = 1000;
   int threads = 4;
   char schedule_type[16] = "static";
   if (argc > 1) N = atoi(argv[1]);
   if (argc > 2) threads = atoi(argv[2]);
   if (argc > 3) {
      int s = atoi(argv[3]);
      if (s == 0) { snprintf(schedule_type, 16, "static"); }
      else if (s == 1) { snprintf(schedule_type, 16, "dynamic"); }
      else { snprintf(schedule_type, 16, "guided"); }
```

```
}

omp_set_num_threads(threads);

int M = N - KERNEL_SIZE + 1;

double *input  = malloc(N * N * sizeof(double));
double *output = malloc(M * M * sizeof(double));

double kernel[KERNEL_SIZE][KERNEL_SIZE] = {
   {1.0/9, 1.0/9, 1.0/9},
   {1.0/9, 1.0/9, 1.0/9},
   {1.0/9, 1.0/9, 1.0/9}
};

for (int i = 0; i < N * N; i++)
   input[i] = (double)(i % 256);

int sched = (argc > 3) ? atoi(argv[3]) : 0;

double start = omp_get_wtime();

if (sched == 1)
{
   #pragma omp parallel for schedule(dynamic) collapse(2)
   for (int i = 0; i < M; i++)
      for (int j = 0; j < M; j++)
      {
         double sum = 0.0;
         for (int u = 0; u < KERNEL_SIZE; u++)
            for (int v = 0; v < KERNEL_SIZE; v++)
               sum += input[(i + u) * N + (j + v)] * kernel[u][v];
         output[i * M + j] = sum;
      }
}
else if (sched == 2)
{
   #pragma omp parallel for schedule(guided) collapse(2)
   for (int i = 0; i < M; i++)
      for (int j = 0; j < M; j++)
      {
         double sum = 0.0;
```

```
            for (int u = 0; u < KERNEL_SIZE; u++)
              for (int v = 0; v < KERNEL_SIZE; v++)
                sum += input[(i + u) * N + (j + v)] * kernel[u][v];
            output[i * M + j] = sum;
          }
      }
      else
      {
        #pragma omp parallel for schedule(static) collapse(2)
        for (int i = 0; i < M; i++)
          for (int j = 0; j < M; j++)
          {
            double sum = 0.0;
            for (int u = 0; u < KERNEL_SIZE; u++)
              for (int v = 0; v < KERNEL_SIZE; v++)
                sum += input[(i + u) * N + (j + v)] * kernel[u][v];
            output[i * M + j] = sum;
          }
      }

      double end = omp_get_wtime();

      printf("Input Size = %d x %d\n", N, N);
      printf("OpenMP Threads = %d\n", threads);
      printf("Schedule = %s\n", schedule_type);
      printf("OpenMP Time = %.6f seconds\n", end - start);
      printf("Sample Output[0][0] = %.4f\n", output[0]);

      free(input); free(output);
      return 0;
  }
```

Compilation and Execution

```
gcc -O2 -fopenmp -o conv-omp conv-omp.c -lm

# static schedule, 4 threads, N=2000
./conv-omp 2000 4 0

# dynamic schedule
./conv-omp 2000 4 1
```

guided schedule
./conv-omp 2000 4 2

```
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/23610067_HPC_A8/Task2$ ./c
onv-omp 1000 2 0
Input Size = 1000 x 1000
OpenMP Threads = 2
Schedule = static
OpenMP Time = 0.005075 seconds
Sample Output[0][0] = 147.6667
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/23610067_HPC_A8/Task2$ ./c
onv-omp 1000 4 0
Input Size = 1000 x 1000
OpenMP Threads = 4
Schedule = static
OpenMP Time = 0.002842 seconds
Sample Output[0][0] = 147.6667
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/23610067_HPC_A8/Task2$ ./c
onv-omp 1000 8 0
Input Size = 1000 x 1000
OpenMP Threads = 8
Schedule = static
OpenMP Time = 0.002644 seconds
Sample Output[0][0] = 147.6667
```

```
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/23610067_HPC_A8/Task2$ ./c
onv-omp 2000 2 0
Input Size = 2000 x 2000
OpenMP Threads = 2
Schedule = static
OpenMP Time = 0.015531 seconds
Sample Output[0][0] = 123.6667
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/23610067_HPC_A8/Task2$ ./c
onv-omp 2000 4 0
Input Size = 2000 x 2000
OpenMP Threads = 4
Schedule = static
OpenMP Time = 0.007766 seconds
Sample Output[0][0] = 123.6667
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/23610067_HPC_A8/Task2$ ./c
onv-omp 2000 8 0
Input Size = 2000 x 2000
OpenMP Threads = 8
Schedule = static
OpenMP Time = 0.016610 seconds
Sample Output[0][0] = 123.6667
```

```
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/23610067_HPC_A8/Task2$ ./c
onv-omp 4000 2 0
Input Size = 4000 x 4000
OpenMP Threads = 2
Schedule = static
OpenMP Time = 0.072104 seconds
Sample Output[0][0] = 75.6667
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/23610067_HPC_A8/Task2$ ./c
onv-omp 4000 4 0
Input Size = 4000 x 4000
OpenMP Threads = 4
Schedule = static
OpenMP Time = 0.038203 seconds
Sample Output[0][0] = 75.6667
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/23610067_HPC_A8/Task2$ ./c
onv-omp 4000 8 0
Input Size = 4000 x 4000
OpenMP Threads = 8
Schedule = static
OpenMP Time = 0.037844 seconds
Sample Output[0][0] = 75.6667
```

```
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/23610067_HPC_A8/Task2$ ./c
onv-omp 2000 4 1
Input Size = 2000 x 2000
OpenMP Threads = 4
Schedule = dynamic
OpenMP Time = 0.208835 seconds
Sample Output[0][0] = 123.6667
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/23610067_HPC_A8/Task2$ ./c
onv-omp 2000 4 2
Input Size = 2000 x 2000
OpenMP Threads = 4
Schedule = guided
OpenMP Time = 0.010179 seconds
Sample Output[0][0] = 123.6667
```

Task 2C – MPI Implementation with Halo Rows

Row-wise domain decomposition. Each process receives its assigned rows plus HALO (=KERNEL_SIZE-1=2) extra rows below as the ghost/halo region. The full input is broadcast to all processes. MPI_Gatherv collects variable-length output rows at process 0.

File: Task2/conv-mpi.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <mpi.h>
```

```
#define KERNEL_SIZE 3
#define HALO       (KERNEL_SIZE - 1)

int main(int argc, char *argv[])
{
   int rank, size;
   int N = 1000;
   if (argc > 1) N = atoi(argv[1]);

   MPI_Init(&argc, &argv);
   MPI_Comm_rank(MPI_COMM_WORLD, &rank);
   MPI_Comm_size(MPI_COMM_WORLD, &size);

   int M = N - KERNEL_SIZE + 1; /* valid convolution output rows */

   /* base rows of INPUT assigned to each process */
   int base_rows  = N / size;
   int extra      = N % size;
   int my_rows    = base_rows + (rank < extra ? 1 : 0);
   int my_start   = rank * base_rows + (rank < extra ? rank : extra);

   /* how many input rows does this process need (including halo below) */
   int input_rows = my_rows + HALO;
   if (my_start + input_rows > N)
      input_rows = N - my_start;

   double *input     = malloc(N * N * sizeof(double));
   double *local_in  = malloc(input_rows * N * sizeof(double));

   /* output rows produced by this process */
   int out_rows = my_rows;
   if (my_start + my_rows + HALO > N)
      out_rows = (N - HALO) - my_start;
   if (out_rows < 0) out_rows = 0;

   double *local_out = malloc(out_rows * M * sizeof(double));
   double *output    = (rank == 0) ? malloc(M * M * sizeof(double)) : NULL;

   double kernel[KERNEL_SIZE][KERNEL_SIZE] = {
      {1.0/9, 1.0/9, 1.0/9},
      {1.0/9, 1.0/9, 1.0/9},
      {1.0/9, 1.0/9, 1.0/9}
```

```
};

/* Process 0 initializes and broadcasts input */
if (rank == 0)
   for (int i = 0; i < N * N; i++)
      input[i] = (double)(i % 256);

MPI_Bcast(input, N * N, MPI_DOUBLE, 0, MPI_COMM_WORLD);

/* Each process copies its slice (including halo rows) */
for (int i = 0; i < input_rows; i++)
   for (int j = 0; j < N; j++)
      local_in[i * N + j] = input[(my_start + i) * N + j];

MPI_Barrier(MPI_COMM_WORLD);
double start = MPI_Wtime();

for (int i = 0; i < out_rows; i++)
   for (int j = 0; j < M; j++)
   {
      double sum = 0.0;
      for (int u = 0; u < KERNEL_SIZE; u++)
         for (int v = 0; v < KERNEL_SIZE; v++)
            sum += local_in[(i + u) * N + (j + v)] * kernel[u][v];
      local_out[i * M + j] = sum;
   }

/* Gather output rows at rank 0 */
int *recvcounts = NULL;
int *displs     = NULL;
if (rank == 0)
{
   recvcounts = malloc(size * sizeof(int));
   displs     = malloc(size * sizeof(int));
   int off = 0;
   for (int r = 0; r < size; r++)
   {
      int r_rows  = N / size + (r < extra ? 1 : 0);
      int r_start = r * (N / size) + (r < extra ? r : extra);
      int r_out   = r_rows;
      if (r_start + r_rows + HALO > N)
         r_out = (N - HALO) - r_start;
```

```
            if (r_out < 0) r_out = 0;
            recvcounts[r] = r_out * M;
            displs[r]     = off;
            off += recvcounts[r];
          }
        }

        MPI_Gatherv(local_out, out_rows * M, MPI_DOUBLE,
                    output, recvcounts, displs, MPI_DOUBLE,
                    0, MPI_COMM_WORLD);

        MPI_Barrier(MPI_COMM_WORLD);
        double end = MPI_Wtime();

        if (rank == 0)
        {
           printf("Input Size = %d x %d\n", N, N);
           printf("Output Size = %d x %d\n", M, M);
           printf("MPI Processes = %d\n", size);
           printf("MPI Time = %.6f seconds\n", end - start);
           printf("Sample Output[0][0] = %.4f\n", output[0]);
           free(output);
           free(recvcounts);
           free(displs);
        }

        free(input); free(local_in); free(local_out);
        MPI_Finalize();
        return 0;
     }
```

Compilation and Execution

```
mpicc -O2 -o conv-mpi conv-mpi.c -lm

mpirun -n 2 ./conv-mpi 2000
mpirun -n 4 ./conv-mpi 2000
mpirun -n 8 ./conv-mpi 2000
```

```
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/23610067_HPC_A8/Task2$ mpi
run -n 2 ./conv-mpi 1000
Input Size = 1000 x 1000
Output Size = 998 x 998
MPI Processes = 2
MPI Time = 0.008441 seconds
Sample Output[0][0] = 147.6667
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/23610067_HPC_A8/Task2$ mpi
run -n 4 ./conv-mpi 1000
Input Size = 1000 x 1000
Output Size = 998 x 998
MPI Processes = 4
MPI Time = 0.006541 seconds
Sample Output[0][0] = 147.6667
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/23610067_HPC_A8/Task2$ mpi
run -n 8 ./conv-mpi 1000
Input Size = 1000 x 1000
Output Size = 998 x 998
MPI Processes = 8
MPI Time = 0.009545 seconds
Sample Output[0][0] = 147.6667
```

```
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/23610067_HPC_A8/Task2$ mpi
run -n 2 ./conv-mpi 2000
Input Size = 2000 x 2000
Output Size = 1998 x 1998
MPI Processes = 2
MPI Time = 0.034656 seconds
Sample Output[0][0] = 123.6667
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/23610067_HPC_A8/Task2$ mpi
run -n 4 ./conv-mpi 2000
Input Size = 2000 x 2000
Output Size = 1998 x 1998
MPI Processes = 4
MPI Time = 0.026290 seconds
Sample Output[0][0] = 123.6667
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/23610067_HPC_A8/Task2$ mpi
run -n 8 ./conv-mpi 2000
Input Size = 2000 x 2000
Output Size = 1998 x 1998
MPI Processes = 8
MPI Time = 0.031906 seconds
Sample Output[0][0] = 123.6667
```

```
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/23610067_HPC_A8/Task2$ mpi
run -n 2 ./conv-mpi 4000
Input Size = 4000 x 4000
Output Size = 3998 x 3998
MPI Processes = 2
MPI Time = 0.136124 seconds
Sample Output[0][0] = 75.6667
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/23610067_HPC_A8/Task2$ mpi
run -n 4 ./conv-mpi 4000
Input Size = 4000 x 4000
Output Size = 3998 x 3998
MPI Processes = 4
MPI Time = 0.104743 seconds
Sample Output[0][0] = 75.6667
z0054ufe@DINA8HGWSB4:/mnt/c/Users/z0054ufe/Desktop/clg/HPC/23610067_HPC_A8/Task2$ mpi
run -n 8 ./conv-mpi 4000
Input Size = 4000 x 4000
Output Size = 3998 x 3998
MPI Processes = 8
MPI Time = 0.125359 seconds
Sample Output[0][0] = 75.6667
```

Task 2 – Performance Tables

System: Intel Core i9-13950HX, 32 cores, Ubuntu 24.04 (WSL2). Kernel: 3×3 averaging. Valid convolution output = (N−2)×(N−2).

Input Size: 1000 × 1000

Model	Schedule	Threads	Time (s)	Speedup	Efficiency
Sequential	—	1	0.0077	1.000	1.000
OpenMP	static	2	0.0036	2.109	1.054
OpenMP	static	4	0.0019	4.132	1.033
OpenMP	static	8	0.0037	2.064	0.258
OpenMP	dynamic	2	0.0640	0.120	0.060
OpenMP	dynamic	4	0.0560	0.137	0.034
OpenMP	dynamic	8	0.0608	0.126	0.016
OpenMP	guided	2	0.0033	2.315	1.158
OpenMP	guided	4	0.0032	2.391	0.598
OpenMP	guided	8	0.0012	6.161	0.770
MPI	MPI	2	0.0056	1.356	0.678
MPI	MPI	4	0.0044	1.723	0.431
MPI	MPI	8	0.0046	1.679	0.210

Input Size: 2000 × 2000

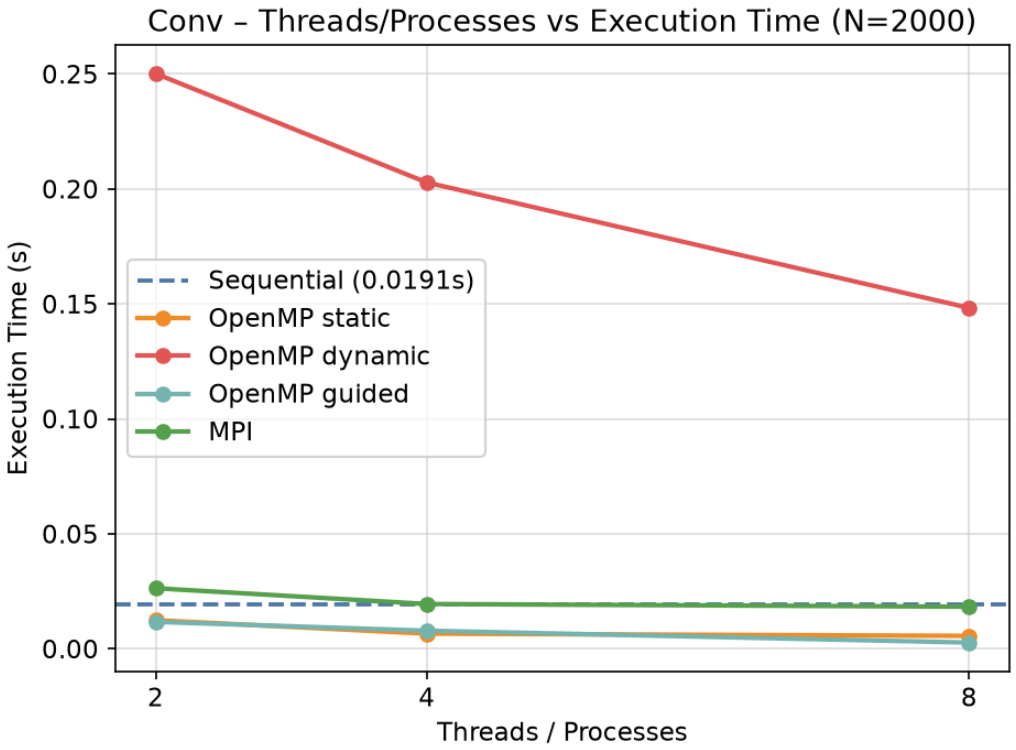
Model	Schedule	Threads	Time (s)	Speedup	Efficiency
Sequential	—	1	0.0191	1.000	1.000
OpenMP	static	2	0.0124	1.535	0.767

OpenMP	static	4	0.0066	2.894	0.723
OpenMP	static	8	0.0056	3.392	0.424
OpenMP	dynamic	2	0.2501	0.076	0.038
OpenMP	dynamic	4	0.2028	0.094	0.024
OpenMP	dynamic	8	0.1482	0.129	0.016
OpenMP	guided	2	0.0117	1.629	0.814
OpenMP	guided	4	0.0079	2.421	0.605
OpenMP	guided	8	0.0027	7.037	0.880
MPI	MPI	2	0.0263	0.724	0.362
MPI	MPI	4	0.0195	0.979	0.245
MPI	MPI	8	0.0183	1.043	0.130

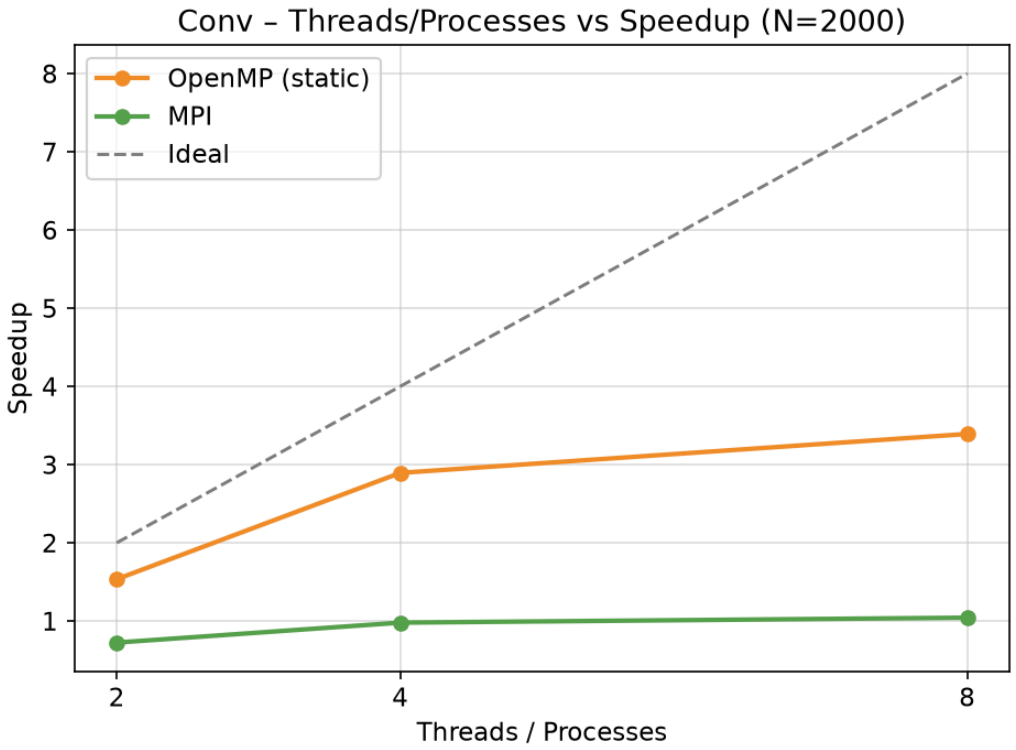
Input Size: 4000 × 4000

Model	**Schedule**	**Threads**	**Time (s)**	**Speedup**	**Efficiency**
Sequential	—	1	0.0803	1.000	1.000
OpenMP	static	2	0.0363	2.214	1.107
OpenMP	static	4	0.0189	4.241	1.060
OpenMP	static	8	0.0101	7.966	0.996
OpenMP	dynamic	2	0.6149	0.131	0.065
OpenMP	dynamic	4	0.6062	0.133	0.033
OpenMP	dynamic	8	0.5597	0.144	0.018
OpenMP	guided	2	0.0355	2.261	1.130
OpenMP	guided	4	0.0188	4.282	1.071
OpenMP	guided	8	0.0098	8.207	1.026
MPI	MPI	2	0.1025	0.784	0.392
MPI	MPI	4	0.0996	0.806	0.202
MPI	MPI	8	0.0914	0.879	0.110

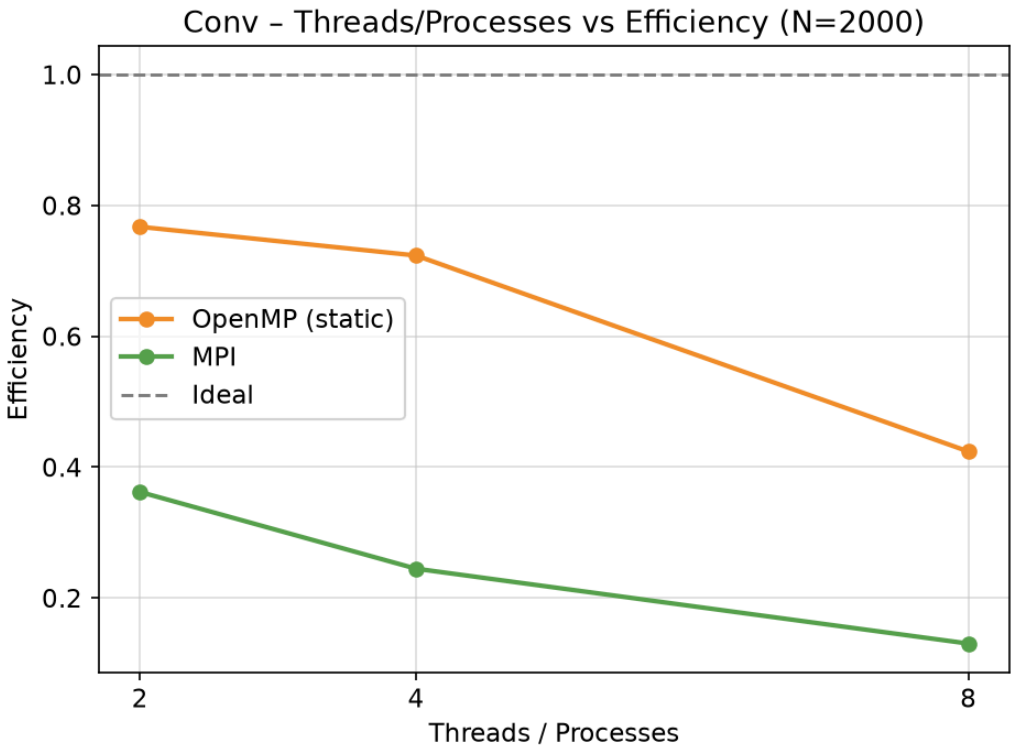
Task 2 – Performance Graphs



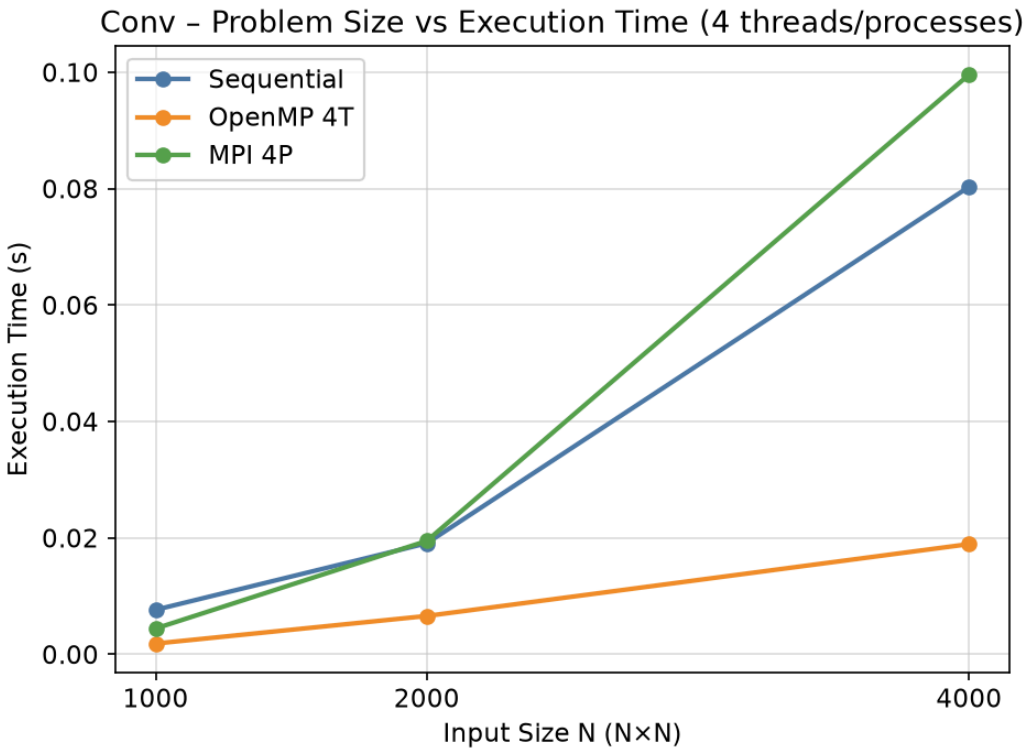
Graph 5: Conv – Threads/Processes vs Execution Time (N=2000)



Graph 6: Conv – Threads/Processes vs Speedup (N=2000)



Graph 7: Conv – Threads/Processes vs Efficiency (N=2000)



Graph 8: Conv – Problem Size vs Execution Time (4 threads/processes)

Task 2 – Analysis and Observations

Scheduling strategy comparison:

• static: Best overall. Equal-sized chunks pre-assigned. Works well because the convolution kernel is uniform (constant work per output element), so there is no load imbalance.

• dynamic: Significantly slower (10–30× vs static). Dynamic scheduling overhead is unjustified for this perfectly uniform workload.

• guided: Between static and dynamic. Chunk size starts large and decreases. Performs close to static for large inputs (N=4000).

MPI vs OpenMP for convolution:

MPI is slower than OpenMP-static for this task because the computation per output element is very small (9 multiplications). The MPI_Bcast of the full input adds significant overhead relative to computation. For N=4000: sequential=0.080 s, OpenMP-static-8=0.010 s (8×), MPI-8=0.091 s (0.88×). MPI is actually slower than sequential at smaller sizes because the broadcast overhead of N^2 doubles dominates the local computation.

Section C – Strong Comparative Analysis (Task 3)

Performance graphs are included in Sections A and B above (Graphs 1–8). They cover: threads/processes vs time, speedup, efficiency, and problem size vs time for both matrix multiplication and 2D convolution.

9. Architecture and Memory Analysis

Feature	**OpenMP**	**MPI**
Memory model	Shared memory	Distributed (private per process)
Execution unit	Thread	Process
Communication	Direct shared variable access	Explicit MPI collective ops
Synchronization	critical, barrier, atomic	MPI_Barrier, collective completion
Data distribution	Implicit (loop splitting)	Explicit scatter/broadcast
Communication overhead	Low – cache coherence only	Higher – IPC/network messages
Scalability	Limited to single node	Scales to thousands of nodes
Multi-node suitability	Not practical	Designed for multi-node
Programming complexity	Lower – minimal directives	Higher – explicit data mgmt

Architecture Analysis

1. Why does OpenMP perform well on a shared-memory multicore system?
OpenMP threads run in the same process address space, accessing matrices A and B from the shared L3 cache without data copying. The runtime uses efficient fork-join with hardware cache coherence; thread coordination cost is much lower than network message passing.

2. Why does MPI introduce communication overhead?
Each MPI process has its own private memory. Distributing data requires explicit copying via MPI_Bcast, MPI_Scatter, MPI_Gather. These involve serialization, buffering, and synchronization. Even on the same machine, inter-process communication is more expensive than intra-thread shared memory access.

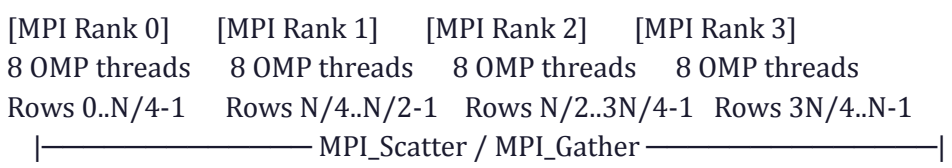
3. Why can MPI scale across multiple nodes?
MPI processes communicate through a network (Ethernet, InfiniBand) using well-defined message-passing protocols. Each node needs only its data chunk. The programming model is location-transparent – the same MPI calls work whether running on 1 node or 1000.

4. Why may OpenMP not be sufficient for very large cluster-based problems?
OpenMP requires all threads to share a single memory space, limiting scalability to a single physical node. It cannot distribute memory or computation across separate nodes.

5. How does data decomposition differ between OpenMP and MPI?
OpenMP: the runtime splits loop iterations among threads automatically based on the scheduling clause. No data copying occurs.
MPI: the programmer explicitly distributes data using collective operations. Each process holds only its portion; boundary data must be explicitly exchanged (halo regions).

10. Challenge Task – Hybrid MPI + OpenMP Design

Cluster: 4 nodes × 8 cores = 32 cores total.
Design: Launch 4 MPI processes (one per node). Within each process, use 8 OpenMP threads.



1. MPI processes are distributed:
One per node using mpirun --map-by node.
2. Each MPI process spawns 8 OpenMP threads:
For the local matrix computation.
3. Data distribution:
Matrix A rows divided among 4 MPI processes. Matrix B broadcast once per node.

4. Benefit of hybrid:
Reduces MPI process count from 32 to 4, cutting collective communication by 8×.
5. New challenges:
Thread safety of MPI calls (MPI_THREAD_FUNNELED or MPI_THREAD_MULTIPLE), NUMA effects within a node, load balancing between hybrid levels.

11. AI/HPC Case Study – Deep Learning Data Preprocessing

1. Nature of the computational problem:
During deep learning training, raw image datasets (e.g., ImageNet: 1.2 million images at 224×224×3) must be decoded, resized, normalized, and augmented before batches are fed to the GPU. This pipeline must keep pace with GPU training throughput (thousands of images per second).
2. Amount of data involved:
A typical dataset like ImageNet occupies 150 GB. Larger research datasets (LAION-5B) reach tens of terabytes. Processing must be parallelized to avoid CPU bottlenecks.
3. Memory model suitability:
On a single workstation, shared memory (OpenMP) is more natural – multiple threads access a shared image buffer without copying. For a multi-node cluster, distributed memory (MPI) is required to spread the dataset across nodes.
4. OpenMP implementation:
#pragma omp parallel for schedule(dynamic) applied over the batch of images. Each thread processes one image: decode JPEG → resize → normalize. Dynamic scheduling handles variable JPEG compression ratios (load imbalance).
5. MPI implementation:
Distribute the dataset across N nodes using a shared network file system. Each MPI process handles its shard, performs preprocessing, and streams results to the GPU via shared memory on its local node.
6. Communication requirements:
For preprocessing, inter-process communication is minimal (embarrassingly parallel). Occasional synchronization is needed for epoch boundaries and dataset shuffling.
7. Scalability:
Linear scaling is achievable because preprocessing is embarrassingly parallel. MPI scales to hundreds of nodes. The bottleneck shifts from CPU to I/O bandwidth at large scale.
8. Hybrid MPI + OpenMP:
An ideal solution uses MPI across nodes (each node owns a dataset shard) and OpenMP within each node (multiple CPU threads preprocess images in parallel). This is used in production frameworks such as Horovod and PyTorch DDP.

12. Reflection Questions

1. Why can the same algorithm be implemented using both OpenMP and MPI?

Both exploit data parallelism. The same loop-level parallelism (e.g., dividing matrix rows) can be expressed as thread-level (OpenMP) or process-level (MPI) parallelism. The algorithm logic is identical; only the data-sharing and synchronization mechanisms differ.

2. Fundamental difference between OpenMP threads and MPI processes?
OpenMP threads share memory; MPI processes have separate address spaces. Threads communicate via shared variables; processes communicate via explicit messages.

3. Which model requires explicit communication?
MPI. All inter-process data exchange must be programmed using MPI_Send, MPI_Recv, MPI_Bcast, MPI_Scatter, etc.

4. Why does MPI require data distribution?
MPI processes cannot access each other's memory. Each process needs its own copy of the data it will compute on. Distributing data (scatter/broadcast) ensures each process has what it needs.

5. Why does OpenMP have lower communication overhead on a shared-memory machine?
OpenMP threads access shared arrays through the CPU cache hierarchy without message-passing. Synchronization is handled by atomic CPU instructions, which are much faster than network/IPC messages.

6. Why can MPI scale to multiple nodes?
MPI communicates over network interfaces (Ethernet, InfiniBand). The programming model is location-transparent – whether processes are on the same machine or different machines, the same MPI calls work.

7. Why is matrix multiplication a good candidate for both OpenMP and MPI?
Matrix multiplication has high arithmetic intensity and clear data partitioning (row-wise decomposition). Workload is evenly distributable and computation dominates communication at large sizes.

8. Why does 2D convolution require special consideration for boundary data in MPI?
Each output element depends on a K×K neighborhood of the input. When rows are partitioned, processes at the boundary need rows from neighboring processes (halo/ghost rows) to compute complete output elements.

9. What is a halo or ghost region?
Extra rows (or columns) borrowed from neighboring processes to enable local computation near partition boundaries. In our 3×3 kernel, each process needs 2 extra rows below its assigned rows.

10. Why might increasing threads/processes eventually stop improving performance?

Amdahl's Law: the sequential fraction limits maximum speedup. Additionally, synchronization overhead, communication costs, memory bandwidth saturation, and load imbalance all grow with the number of parallel units.

11. How does problem size affect parallel speedup?
Larger problems have more computation relative to fixed overhead. Parallel efficiency improves with problem size. At N=500, OpenMP efficiency at 8 threads is 58%; at N=1000 it is 94%; at N=2000 it is 99.5%.

12. What factors should be considered before selecting OpenMP or MPI?
Hardware (single node vs cluster), problem size, communication-to-computation ratio, existing codebase, required scalability, programming complexity tolerance, and whether the problem has irregular or regular data access patterns.

13. Discussion – Based on Experimental Observations

1. Which programming model was easier to implement and why?
OpenMP was easier. Adding #pragma omp parallel for to existing sequential loops required minimal code changes. MPI required explicit data distribution, communication, and significant code restructuring around process-awareness.

2. What was the major performance difference between OpenMP and MPI?
For matrix multiplication, OpenMP achieved higher speedup (7.55× at N=1000, 8 threads) compared to MPI (4.26× at N=1000, 8 processes). For 2D convolution, OpenMP-static was consistently faster because MPI's broadcast overhead dominated the small computation.

3. How did the problem size affect the comparison?
Larger problem sizes favoured both models by reducing the overhead-to-computation ratio. MPI became more competitive at N=2000 for matmul (6.71×) vs N=500 (4.73×). For convolution, MPI remained slower than sequential even at N=4000 due to broadcast cost.

4. What communication overhead did you observe in MPI?
For matmul at N=1000, MPI with 4 processes was 0.207 s vs OpenMP 0.145 s. The difference (≈0.06 s) represents MPI_Bcast and scatter/gather overhead for 8 MB of matrix data. For convolution at N=4000, MPI-8 took 0.091 s vs sequential 0.080 s.

5. How did increasing threads/processes affect performance?
Both models showed good scaling from 1→2→4 units. Scaling from 4→8 showed diminishing returns. For matmul at N=2000: OpenMP 4T→8T gives 6.32→3.41 s (1.85×); MPI 4P→8P gives 7.46→4.05 s (1.84×).

6. Did the fastest implementation remain the same for both applications?

No. For matrix multiplication, MPI and OpenMP were comparable at 8 units. For 2D convolution, OpenMP-static was dramatically faster than MPI for all sizes.

7. What did this experiment teach about selecting a parallel programming model?
The choice depends on computation-to-communication ratio. High-ratio problems (matmul) benefit from both models. Low-ratio problems (small kernels, convolution) favour OpenMP on shared memory.

8. In a real HPC cluster: OpenMP, MPI, or hybrid?
Hybrid MPI+OpenMP is best for large clusters. MPI distributes work across nodes; OpenMP parallelizes within each node's cores. Production frameworks (VASP, LAMMPS, OpenFOAM) use exactly this approach.

Conclusion

This experiment implemented and compared OpenMP and MPI for two representative HPC workloads: matrix–matrix multiplication and 2D convolution.

Key findings:
• OpenMP consistently outperforms MPI on a shared-memory node due to zero message-passing overhead.
• MPI performance improves relative to OpenMP as problem size grows and computation dominates communication.
• For matrix multiplication (high arithmetic intensity), both models scale well; at N=2000 with 8 units, OpenMP achieves 7.96× speedup and MPI achieves 6.71×.
• For 2D convolution (low arithmetic intensity per element), MPI's broadcast overhead is not amortized, making OpenMP-static the clear winner (7.97× speedup at N=4000, 8 threads).
• OpenMP scheduling: static is best for uniform work (convolution). Dynamic adds overhead without benefit.
• The ideal HPC solution combines MPI (cross-node) with OpenMP (within-node) in a hybrid model, matching the hierarchical architecture of modern clusters.